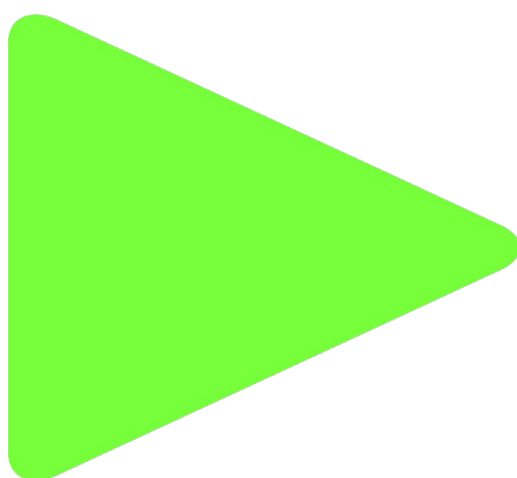


Documentación Técnica VideoClub



Alumno	Maria Emilia Navalón Vidal
Profesor	Juan Antonio Vilaplana Pérez
Asignatura	Desarrollo de Interficies

IES L'ESTACIÓ
2º Desarrollo de Aplicaciones Multiplataforma
2025-2026

ÍNDICE

1. Arquitectura MVVM Utilizada.....	3
Model (Modelo):.....	3
View (Vista):.....	3
ViewModel (Vista-Modelo):.....	3
2. Diagrama de Capas.....	4
3. Descripción de Carpetas del Proyecto.....	5
4. Explicación de los Componentes del Proyecto.....	5
Models (Modelos).....	5
ViewModels (Vista-Modelos).....	5
Commands.....	6
Acceso a Datos.....	6
5. Diagrama de Base de Datos.....	6

1. Arquitectura MVVM Utilizada

La aplicación se ha desarrollado siguiendo el patrón de diseño Model-View-ViewModel (MVVM) que divide la aplicación en tres capas principales:

Model	--	Datos y lógica de negocio
View	--	Interfaz gráfica (XAML)
ViewModel	--	Lógica de presentación y enlace entre View y Model

Model (Modelo):

El Model representa los datos y reglas de negocio de la aplicación. Contiene las clases que mapean las tablas de la base de datos (Clientes, Productos, Alquileres). No debe conocer nada sobre la interfaz de usuario.

- Clases simples (POCO)
- Contienen propiedades y validaciones
- No contienen lógica de UI

View (Vista):

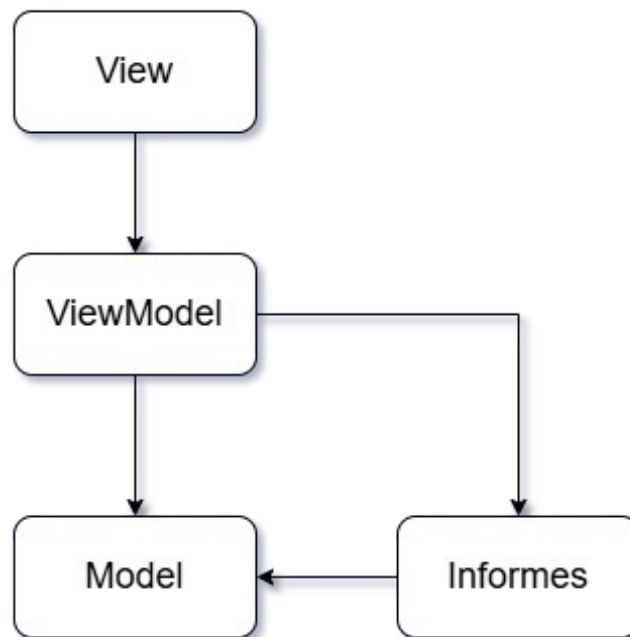
Es la interfaz de usuario (XAML). Su única función es mostrar información al usuario y capturar acciones del usuario.

- No debe contener lógica de negocio
- El code-behind debe estar vacío o mínimo
- Usa Bindings en lugar de código

ViewModel (Vista-Modelo):

El ViewModel actúa como intermediario entre la View y el Model. Contiene la lógica de presentación, las propiedades que la Vista observa y los comandos que responden a las acciones del usuario.

2. Diagrama de Capas



El Data Binding es el mecanismo que permite conectar la interfaz de usuario con las propiedades del ViewModel, sin necesidad de escribir código en el code-behind.

Bindings: Enlazan las propiedades del ViewModel con los controles de la Vista.

Commands (ICommand): Permiten que elementos de la interfaz, como botones, ejecuten métodos lógicos en el ViewModel.

Notificaciones: El ViewModel implementa INotifyPropertyChanged para avisar a la Vista cuando los datos han cambiado y debe actualizarse.

3. Descripción de Carpetas del Proyecto

La estructura de carpetas es la siguiente:

/Model: Contiene las entidades de datos y las reglas de negocio.

/ViewModel: Contiene la lógica de presentación.

/View: Archivos XAML que definen la interfaz gráfica.

/Services: Capa para lógica de negocio adicional y validaciones complejas.

/Repositorios: Encargados del acceso a datos, permitiendo que el ViewModel solicite información.

/Commands: Contiene implementaciones genéricas como RelayCommand para gestionar las acciones de los botones.

/Informes: Dedicada a la generación de reportes utilizando Crystal Reports. Contiene los archivos y la vista para la visualización de datos.

/Test: Contiene clases dedicadas a verificar automáticamente que las validaciones funcionen correctamente.

4. Explicación de los Componentes del Proyecto

Models (Modelos)

Se han implementado clases simples (POCO) que mapean directamente las tablas de la base de datos: Clientes, Productos, Alquileres, Géneros y Tarifas. Estas clases contienen las propiedades y validaciones necesarias, manteniendo la independencia total de la interfaz de usuario.

ViewModels (Vista-Modelos)

Se preparan los datos para que la interfaz pueda consumirlos.

Con la implementación de la interfaz `INotifyPropertyChanged`, el `ViewModel` detecta cambios en las propiedades y notifica a la Vista para que esta se actualice en tiempo real.

Commands

Los comandos sustituyen a los antiguos controladores de eventos para mantener una arquitectura limpia.

La clase `RelayCommand` hace que la lógica de la acción se quede solamente en el `ViewModel`. Un botón en la Vista se enlaza a una propiedad `ICommand`. Al pulsarlo, se ejecuta el método definido en el `ViewModel`.

Acceso a Datos

Se encarga de la persistencia de la información para el resto de la aplicación. Se han implementado las clases `SocioRepositorio`, `ActividadRepositorio`, `ReservasRepositorio` para centralizar todas las consultas CRUD.

5. Diagrama de Base de Datos

